# Week 11: Tools, agents and MCP

AI Development Course

Let the model use your code instead of guessing

Tested with anthropic 1.13.0 and claude-haiku-5-5 (real calls), mcp 2.3.0 with Python 3.12, and Claude Code 2.1.296's claude mcp commands (not signed in), 13 October 2026. A signed-in Claude Code session, GitHub, and Mac and Windows laptops are being tested.

## Today

- Tool use: the model asks, your code runs
- An agent: that, in a loop, with a limit
- The deal screener's calculator as a tool
- Checking every input the model sends
- One safe tool in your project
- MCP: your tool, inside Claude Code
- Review, Docker, pull request

## The tool cycle

1. Your program sends the question **and** the tool's description
2. The model replies: "please run `percent_change` with old 1500, new 1700"
3. **Your program** runs it and sends back `13.33`
4. The model answers with your number

The model never runs anything itself.

## What goes back and forth

Claude's request (`stop_reason: "tool_use"`):

```
{"type": "tool_use", "id": "toolu_...", "name": "percent_change",
 "input": {"old": 1500, "new": 1700}}
```

Your reply (first in the next user message):

```
{"type": "tool_result", "tool_use_id": "toolu_...", "content": "13.33"}
```

Two calls, both paid: about **$0.00016** in our run.

## An agent: the cycle in a loop

1. Send the question, the tool, the conversation so far
2. `end_turn`? Print the answer. Done.
3. `tool_use`? **Check the inputs**, run the tool, add the result, go to 1
4. Gone round 5 times? Stop and say so

The limit of 5 is the course's rule: Anthropic's docs don't set one.

## Let code do the maths, let the model talk

The deal screener's one tool: `screen_listing`, the week 5 calculator.

```
Use the screen_listing tool for every number you report;
never work numbers out yourself.
```

Our run: `Monthly cash flow: $430.53`, exactly as in week 5. 2 turns, about **$0.00080**.

## What the live runs taught

- First prompt: Claude said the calculator **left out** vacancy and repairs. (It doesn't.)
- Second: it rounded $430.53 to "about $431" and did its own sums.
- Fix: the tool's description names what's included; the prompt says "give the numbers exactly".

**Test your prompt with real runs, and read the answers.**

# Never trust the inputs

The model can send anything. `screen_listing` checks every field:

| Sent | Result |
|---|---|
| a missing field, words, a negative | refused, with the field named |
| a 250% rate, 30.5 years | refused |
| an interest rate of `0.07` | refused: "percentages are written as 7 for 7%" |

Refusals go back as `is_error: true`, so the model can fix it or ask you.

## Missing numbers? Ask, don't guess

```
If the question is missing any of the nine numbers,
ask the user for them and do not call the tool.
```

Our run: it listed the seven missing numbers and called no tool (about **$0.00031**).

# Safe first tools

| Good this week | Not this week |
| --- | --- |
| A calculator you already have | Delete, send, pay, post, write files |
| Looking up your own course-safe data | Other people's accounts or data |
| Fixed conversions | Running code the model writes |

Week 13 is planned to cover what can go wrong with more powerful tools.

## MCP in one picture, in words

**Claude Code** speaks **MCP** to **your MCP server**, which calls **your function**.

- MCP: a standard way to package tools for AI apps
- Write the tool once; Claude Code can use it while it works

# The server

```
from mcp.server import MCPServer

mcp = MCPServer("dealscreener")

@mcp.tool()
def screen_listing(...):
    ...same checks, same calculator...
```

- `mcp[cli]==2.3.0`, Python **3.11 or newer**
- Old tutorials say `FastMCP`: that name is gone in 2.3.0

# Connect it to Claude Code

```
claude mcp add dealscreener --scope project -e PYTHONPATH="$PWD" -- "$PWD/.venv/bin/python" -m dealscreener.mcp_server
claude mcp list
```

- In a **new window with no key loaded**: Claude Code's servers would get it
- `--scope project`: saved in `.mcp.json`, this project only
- `-e PYTHONPATH="$PWD"`: Python can find your code from anywhere
- Full path to the `.venv`'s Python: saved exactly as typed
- "Pending approval": approve it when you start `claude` (being tested)

## Trust before you connect

Claude Code's security page:

> "We encourage either writing your own MCP servers or using MCP servers from providers that you trust."

Anthropic "does not security-audit or manage any MCP server".

This week: connect only the server **you** wrote.

## Your tool, step by step

1. Branch `tool` first
2. Pick a read-or-calculate tool
3. `/superpowers:writing-plans` with the guardrails, then build
4. Ask three questions: answerable, missing input, bad input
5. Tests pass with `.env` moved out
6. MCP server, `claude mcp add`, a screenshot of Claude Code calling it

## Tests that never call Anthropic

- A fake model that **asks for the tool**
- A bad input, and two tool calls in one reply
- Hitting the 5-turn limit
- One live test, only with `RUN_LIVE_TESTS=1`
- The deal screener: 184 tests

# The assignment

| Criterion | Points |
|---|---|
| Merged PR: read-or-calculate tool, capped loop, checked inputs, fake-model tests, no key | 4 |
| MCP server connected: command, screenshot, one agent run's output | 3 |
| Reflection: when to call a tool, and what could go wrong with a tool that changes things | 3 |

Next week: evaluation.